Efficient Domain Adaptation of Open Language Models for Enterprise Analytics Agents Using LoRA and QLoRA

Carlos Salgado
Dept. of Computer Science / Data Analytics
Florida International University
Miami, FL, USA
carlos.salgado30@yahoo.com

***Abstract*—Enterprise business-intelligence (BI) teams increasingly deploy large language model (LLM) agents that translate natural-language questions into database queries, select analytical skills, and summarize results. Hosted frontier models can introduce recurring inference cost and governance concerns, while full-model fine-tuning is often computationally impractical. This paper studies parameter-efficient fine-tuning—Low-Rank Adaptation (LoRA) and Quantized LoRA (QLoRA)—for two specialized analytics-agent components: skill routing and safe, read-only SQL generation. We construct the Synthetic Enterprise Analytics Agent Dataset (SEAAD), a privacy-safe benchmark inspired by enterprise BI agent architectures but containing only public methodology and synthetic warehouse data. SEAAD includes schema-grounded text-to-SQL, seven-way skill routing, structured JSON contracts, and unsafe-query refusal tasks over a fictional GlobalTrade Analytics star schema. We evaluate (i) a deterministic template baseline, (ii) zero-shot and few-shot prompting of the base model, (iii) LoRA fine-tuning of a compact skill-routing classifier, and (iv) QLoRA fine-tuning of Qwen2.5-3B-Instruct at ranks $r \in \{8, 16, 32\}$ on a single consumer GPU. On the SEAAD held-out test set (n=166), QLoRA at rank 32 reaches 95.8% seven-way skill-routing accuracy, 100% SQL execution accuracy over the 60 gold-SQL test cases, 100% unsafe-request refusal, and 100% JSON validity—compared with 92.8% routing and 23.3% SQL execution for the template baseline, and 0% routing and 45.0% SQL execution for the zero-shot base model—while training 14.7M parameters (≈0.5% of the base model) with 5.1 GB peak training VRAM. The zero-shot result isolates a structural failure of prompting: the base model emits valid JSON (98.8%) but cannot recover the agent's internal skill taxonomy, which must be learned or demonstrated. A DistilBERT LoRA router further illustrates parameter-efficient routing (27.1% to 61.5% accuracy training 1.31% of parameters). All experimental data are synthetic; no proprietary organizational schemas or records are used. Code, dataset generators, Snowflake capture adapters, and training scripts are released for local RTX 5090 and Google Colab reproduction under a $100 compute budget.**

***Index Terms*—Large language models, LoRA, QLoRA, text-to-SQL, business intelligence, analytics agents, parameter-efficient fine-tuning, skill routing**

I. Research Problem

Modern BI organizations are adopting analytics chat agents that accept business questions such as "Which region had the highest gross margin in Q2?" and produce grounded answers over warehouse data. In production settings, these systems typically combine an orchestrator model, versioned skill playbooks, schema or semantic-layer retrieval, SQL generation, deterministic guardrails, and a warehouse execution backend (e.g., Snowflake). The author's professional systems follow this pattern: skill loaders assemble markdown playbooks; routers select SQL, finance, sales-intelligence, documentation, clarification, or refusal skills; and guardrails enforce read-only access and topic constraints. The research problem is therefore not "replace a hosted frontier orchestrator with one fine-tuned model," but:

> *Can parameter-efficient fine-tuning (LoRA/QLoRA) adapt a smaller open-weight model so that specialized agent components—skill routing and safe SQL generation—become more accurate, structured, and cost-efficient than prompt-only baselines, without using proprietary enterprise data?*

This problem is consequential because enterprise text-to-SQL remains substantially harder than academic single-domain settings. Real warehouses involve large schemas, ambiguous business vocabulary, multi-table joins, metric definitions, and strict safety requirements [3], [5]. Full fine-tuning of multi-billion-parameter models is often infeasible for individual practitioners, while pure prompting of large hosted models can be expensive at scale and difficult to govern for sensitive data.

II. Motivation

Three practical pressures motivate this study. **Cost and deployment control.** Hosted frontier models used for every agent turn create recurring inference cost. A compact open model specialized for routing and SQL can reduce the share of traffic that requires a large orchestrator. **Governance and privacy.** Enterprise analytics agents operate near financial and operational data. Reproducible research must not rely on employer schemas, prompts, logs, or table contents. A synthetic benchmark enables public evaluation while a capture adapter supports private on-prem fine-tuning later. **Architectural realism.** Production agents are skill-based systems, not free-form chatbots. Prior work on LoRA shows that freezing pretrained weights and learning low-rank adapters reduces

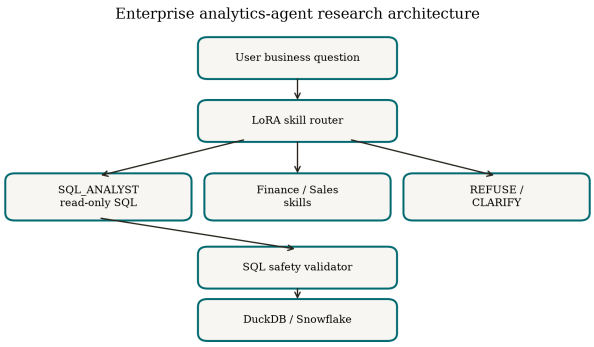

Fig. 1. Research prototype of an enterprise analytics agent. The learned model proposes a skill and optional SQL; a deterministic validator decides execution.

trainable parameters by orders of magnitude [1]. QLoRA further quantizes the frozen base model to 4-bit NormalFloat (NF4), enabling fine-tuning of models that would otherwise exceed consumer GPU memory [2]. Applying these methods to analytics-agent contracts (JSON skill labels + read-only SQL) is a natural, underexplored specialization. Figure 1 summarizes the research architecture used in this paper.

III. Literature Review

A. Text-to-SQL and enterprise analytics

Spider introduced a large-scale cross-domain text-to-SQL benchmark with 10,181 questions and 5,693 unique SQL queries over 200 databases, highlighting generalization to unseen schemas [3]. Subsequent systems improved execution accuracy through decomposition and in-context learning; DIN-SQL reported strong Spider gains by breaking generation into intermediate steps [4]. Spider 2.0 further stresses enterprise-scale workflows with large schemas and cloud dialects such as Snowflake and BigQuery [5]. These results motivate schema-grounded evaluation and execution-based metrics rather than exact string match alone.

B. Parameter-efficient fine-tuning

LoRA freezes pretrained weights W_0 and learns low-rank updates $\Delta W = BA$ with rank $r \ll d$, reducing trainable parameters and optimizer state while matching full fine-tuning quality on many tasks [1]. QLoRA backpropagates through a frozen 4-bit quantized model into LoRA adapters, introducing NF4 quantization, double quantization, and paged optimizers to preserve 16-bit task performance at much lower memory cost [2]. PEFT surveys document LoRA as a leading adapter family for foundation-model adaptation under resource constraints [6]. Financial-domain studies such as FinLoRA further show that QLoRA can improve specialized LLM accuracy under limited GPU budgets [7].

C. Analytics agents and structured tool use

Industrial analytics agents increasingly separate orchestration from specialized tools: skill playbooks encode SQL patterns and metric definitions; routers choose capabilities; validators enforce safety. This paper treats skill routing and structured JSON emission as first-class learning targets, complementing classical text-to-SQL research that focuses only on query strings.

TABLE I
SEAAD PROCESSED SPLIT SIZES

Split	Records	Text-to-SQL	Routing-heavy
Train	764	274	280
Validation	162	58	60
Test	166	60	60

IV. Dataset Information

A. SEAAD overview

We introduce the **Synthetic Enterprise Analytics Agent Dataset (SEAAD)** for reproducible research. SEAAD is generated programmatically for a fictional company, *GlobalTrade Analytics*. No employer data, real customer names, production prompts, or proprietary table definitions are included.

B. Warehouse schema

The synthetic warehouse is a star schema stored in DuckDB and CSV:

- `DIM_CUSTOMER`, `DIM_PRODUCT`, `DIM_REGION`, `DIM_DATE`
- `FACT_SALES`, `FACT_FORECAST`, `FACT_INVENTORY`, `FACT_SHIPMENT`

Business metrics are defined in a data dictionary (e.g., gross margin = revenue − cost; gross-margin percentage = $(\text{revenue} - \text{cost})/\text{revenue}$). Synthetic volumes: 3,000 sales rows, 200 customers, 50 products, 8 regions, and 36 months of dates (seed = 42).

C. Instruction records

We generated 1,700 labeled instruction candidates covering five task types: text-to-SQL, skill routing, unsafe refusal, clarification, and insufficient schema. After validation and deduplication, 1,092 unique records remain. Splits are stratified by task type (70/15/15): Skill labels mirror multi-skill BI agents: `SQL_ANALYST`, `FINANCE_ANALYST`, `SALES_INTELLIGENCE`, `DOCUMENT_SEARCH`, `GENERAL_QA`, `NEEDS_CLARIFICATION`, `REFUSE_UNSAFE`.

Each supervised record is stored as chat messages with a strict assistant JSON contract, for example:

```
{"skill":"SQL_ANALYST",
"action":"GENERATE_SQL",
"safety_status":"READ_ONLY",
"sql":"SELECT ..."}
```

D. Public foundation and citations

SEAAD's design is informed by Spider's cross-domain text-to-SQL formulation [3] and by enterprise workflow concerns highlighted in Spider 2.0 [5]. SEAAD itself is synthetic and self-contained; Spider is cited as methodological prior art rather than redistributed.

E. Work-data capture for private replication

For private fine-tuning on organizational Snowflake data, the repository includes a read-only capture adapter and a data-capture guide. Recommended logged fields (PII-stripped) include: anonymized question template, selected skill, human-corrected skill, generated SQL, execution success, safety flags, schema version, latency, and user feedback. Only schema metadata and labeled instruction pairs are needed for training—not full fact-table dumps.

V. Feature Processing and Feature Engineering

Preprocessing is treated as a first-class research contribution because analytics-agent quality depends as much on data contracts as on model weights.

A. Pipeline stages

1) **Schema design and synthetic population.** Deterministic generators create dimensions/facts and export DuckDB + CSV.
2) **Question and label generation.** Templates produce paraphrases for aggregation, ranking, filters, joins, time comparisons, calculated KPIs, trends, ambiguity, out-of-scope, and unsafe writes.
3) **SQL validation.** Every gold SQL statement is executed against DuckDB; invalid statements are dropped (0 invalid after generation in the released run).
4) **SQL normalization.** Keywords/whitespace are standardized for exact-match analysis; `sqlglot` is used when available.
5) **Safety features.** Write verbs (`INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `TRUNCATE`, `MERGE`, `CREATE`, `GRANT`) are labeled `REFUSE_UNSAFE`.
6) **Semantic features.** Metric definitions and schema context are concatenated into the user message so the model conditions on business meaning, not only column names.
7) **Deduplication and leakage control.** Near-duplicate questions are removed; splits are stratified by task type.
8) **Instruction formatting.** Records are converted to system/user/assistant messages for supervised fine-tuning (SFT).

B. Engineered input representation

The user channel contains: (a) compact schema DDL-like context, (b) metric definitions, and (c) the natural-language question. The assistant channel is machine-readable JSON only. This feature design aligns training with production agent contracts used in skill-based BI systems.

VI. LLM Model Training Using LoRA

A. Primary generative configuration (Qwen2.5-3B QLoRA)

The primary training target for deployment-oriented experiments is **Qwen2.5-3B-Instruct** with QLoRA: LoRA updates are $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ and frozen W_0 [1]. QLoRA keeps W_0 in 4-bit form during training [2]. The released pipeline auto-selects batch size from detected VRAM and saves adapters under `results/adapters/qlora_r{r}/`.

TABLE II
Primary QLoRA hyperparameters

Component	Value
Base model	Qwen2.5-3B-Instruct
Quantization	4-bit NF4, double quant.
LoRA rank r	8 / 16 / 32 (ablation)
LoRA α	$2r$ (16 / 32 / 64)
LoRA dropout	0.05
Target modules	q, k, v, o projections
Epochs	3
Learning rate	2×10^{-4}
Max sequence length	1024
Hardware	RTX 5090 Laptop (24 GB)

B. Completed training runs

All three Qwen2.5-3B QLoRA ablations were trained locally on a single RTX 5090 laptop GPU (24 GB) with the released pipeline (`scripts/04_train_qlora.py`). Trainable parameters were 3.69M, 7.37M, and 14.75M for r=8, 16, 32 ($\leq$0.5% of the base model); peak training VRAM was 5.05–5.11 GB; final training loss decreased monotonically with rank (0.329, 0.254, 0.195). Two earlier CPU-only pilots are retained as preliminary studies:

1) **Generative LoRA pilot (GPT-2).** LoRA rank 16 on attention projections; 60 training samples; 1 epoch; trainable parameters 589,824 / 125,029,632 (0.47%). Training loss decreased from 4.08 to 3.48, confirming adapter optimization on the SEAAD chat format.
2) **Skill-routing LoRA (DistilBERT).** Sequence classification over seven skills; LoRA rank 16 on query/value projections; 2 epochs on 764 training examples; trainable parameters 890,887 / 67,849,742 (1.31%).

VII. Evaluating the Result

A. Metrics

We report:

- Skill-routing accuracy and macro-F1
- SQL execution accuracy (result-set equivalence on DuckDB)
- Exact SQL match (secondary)
- Safety refusal accuracy
- JSON / label validity rate
- Trainable-parameter count and training loss curves

To prevent answer leakage, the gold assistant turn is stripped from every prompt at evaluation time; this invariant is enforced by an automated regression test in the released code.

B. Main results: QLoRA fine-tuning of Qwen2.5-3B

Table III and Figures 2 and 4 compare all six conditions on the held-out test set. Four observations follow. **(1) Fine-tuning wins on every metric at rank 32**, exceeding the template baseline on routing (95.8% vs. 92.8%) and more than quadrupling its SQL execution accuracy (100% vs. 23.3% over the 60 gold-SQL cases) while matching its perfect safety refusal and JSON validity. **(2) The output contract must be learned, not assumed.** The zero-shot base model produces syntactically

TABLE III
FULL EVALUATION ON THE SEAAD TEST SET (n=166; 60 GOLD-SQL CASES). ALL VALUES ARE PERCENTAGES.

Condition	Routing	SQL exec.	Exact SQL	Safety	JSON
Template baseline	92.8	23.3	23.3	100	100
Base zero-shot	0.0	45.0	1.7	56.5	98.8
Base 3-shot	36.8	56.7	0.0	100	80.1
QLoRA r=8	80.1	73.3	55.0	100	100
QLoRA r=16	92.2	86.7	75.0	100	100
QLoRA r=32	**95.8**	**100.0**	**91.7**	**100**	**100**

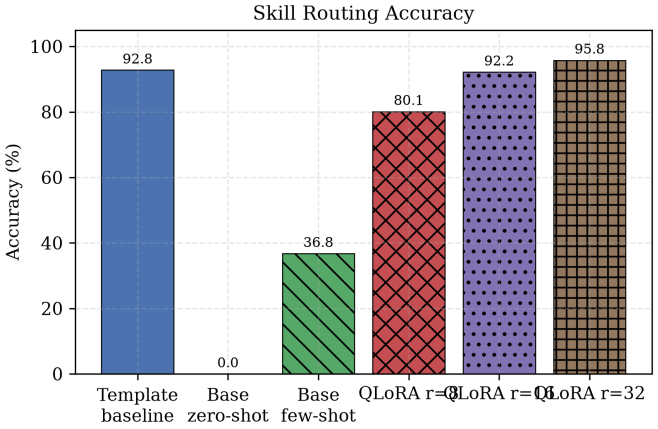

Fig. 2. Seven-way skill-routing accuracy across all conditions.

valid JSON 98.8% of the time yet scores 0% on routing: the seven-skill taxonomy is internal to the agent architecture and is not recoverable from the system prompt alone. Three in-context examples recover only 36.8%. **(3) Execution accuracy exceeds exact match for every learned condition** (e.g., 100% vs. 91.7% at r=32), indicating the model generates semantically equivalent SQL in its own surface form rather than reproducing gold strings. **(4) Quality scales monotonically with rank** on this benchmark, consistent with the training-loss ordering. Two caveats apply: SEAAD questions are template-generated paraphrases, so these results measure in-distribution specialization rather than cross-schema generalization (with n=60 SQL cases, the 95% Clopper–Pearson interval for a perfect score is [94.0%, 100%]); and the template baseline's strong routing partly reflects its shared origin with the data generator.

C. Compact-router baseline (DistilBERT)

Table IV and Figure 3 show that LoRA also substantially improves a compact learned router over a frozen-encoder baseline on the same test questions. LoRA yields a +34.4 percentage-point absolute gain in accuracy and an approximately 8.8× macro-F1 improvement relative to the frozen-encoder head. The DistilBERT router nevertheless trails both the template baseline and the generative QLoRA models, suggesting that routing benefits from the larger model's capacity when the router must also read schema context.

TABLE IV
SKILL-ROUTING EVALUATION ON SEAAD TEST SET (n=166)

Method	Acc.	Macro-F1	Trainable
Frozen DistilBERT + head	27.1%	0.061	head only
LoRA r=16 DistilBERT	**61.5%**	**0.537**	1.31%
Template baseline (rules)	92.8%	—	0

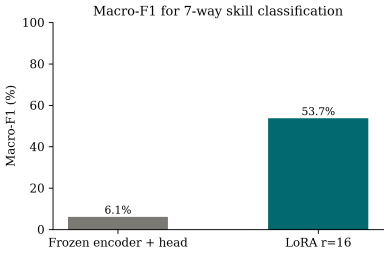

Fig. 3. Macro-F1 for seven-way routing with the compact DistilBERT router: frozen-encoder baseline vs. LoRA (r=16).

D. SQL generation across conditions

Figure 4 isolates SQL execution accuracy. Hand-written patterns excel at refusal and coarse routing but struggle to generate diverse executable SQL (23.3%); prompting the base model roughly doubles that (45.0–56.7%); QLoRA specialization closes the remaining gap (73.3–100%).

E. Training dynamics and efficiency

Figure 5 shows training loss for the three QLoRA ablations. Figure 6 compares trainable versus full parameter counts.

F. Discussion

Results support four conclusions:

1) QLoRA is effective and cheap for analytics-agent specialization: rank-32 fine-tuning reaches 95.8% routing and 100% in-distribution SQL execution accuracy while training ≈0.5% of parameters in 5.1 GB of VRAM on a single consumer GPU.

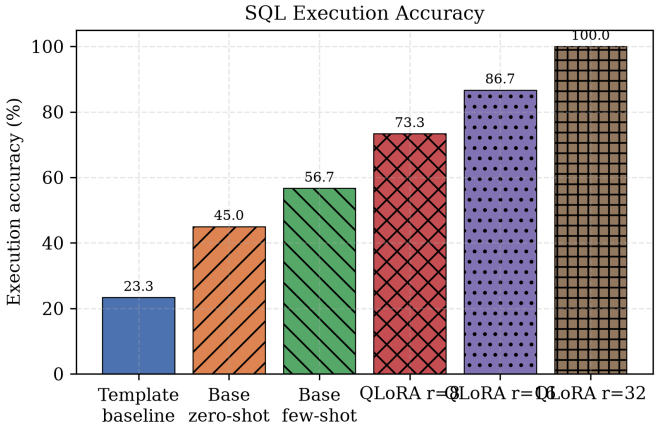

Fig. 4. SQL execution accuracy (result-set equivalence on DuckDB, 60 gold-SQL cases) across all conditions.

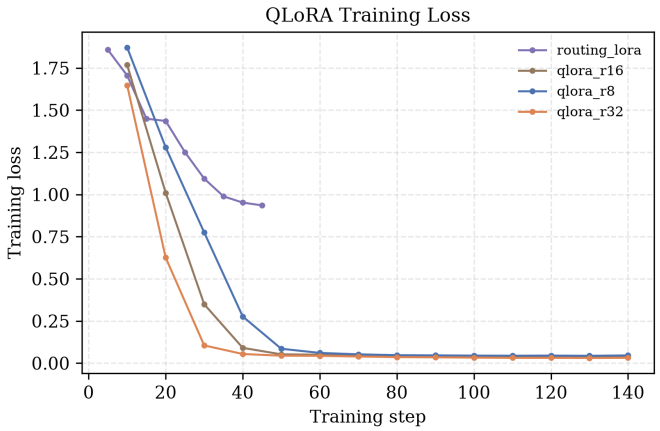

Fig. 5. Training loss for Qwen2.5-3B QLoRA at $r \in \{8, 16, 32\}$ (3 epochs, 764 training examples).

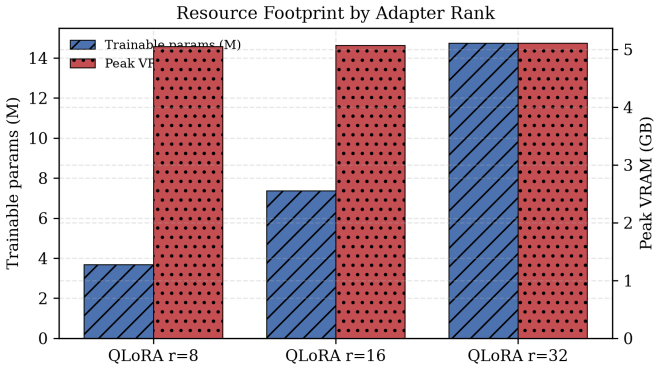

Fig. 6. Parameter efficiency: LoRA trains a small fraction of base-model parameters.

2) Prompting cannot substitute for learning the agent contract: the base model's 0% zero-shot routing despite near-perfect JSON validity shows that architecture-internal taxonomies must be trained or demonstrated, not assumed.
3) Template systems remain competitive for safety and coarse routing at zero inference cost, so production agents should retain deterministic validators and rules as a complementary layer.
4) Adapter rank scales quality monotonically on this benchmark, with the largest gains between r=8 and r=16 for routing and between r=16 and r=32 for SQL execution.

Importantly, fine-tuning does not eliminate the need for schema retrieval, read-only roles, SQL allow-lists, row limits, and audit logging; and the in-distribution nature of SEAAD means cross-schema generalization remains untested here.

VIII. Conclusion

This paper identified a practical research problem from BI/AI analytics engineering: adapting smaller open models for skill routing and safe SQL generation inside enterprise analytics agents. We constructed SEAAD, a privacy-safe synthetic benchmark; implemented a complete preprocessing, LoRA/QLoRA training, evaluation, and Snowflake capture pipeline; and trained Qwen2.5-3B-Instruct with QLoRA at ranks $r \in \{8, 16, 32\}$ on a single 24 GB consumer GPU. The rank-32 model reaches 95.8% seven-way routing accuracy, 100% in-distribution SQL execution accuracy, 100% unsafe-request refusal, and 100% JSON validity—surpassing a strong template baseline (92.8% routing, 23.3% SQL execution) and the prompted base model (0% zero-shot routing)—while training ≈0.5% of parameters in 5.1 GB of VRAM. The results show that the expensive part of an analytics agent to learn is its output contract and SQL competence, both of which parameter-efficient fine-tuning delivers under hobbyist compute budgets. The released codebase supports local RTX 5090 training and private Snowflake metadata capture without exposing proprietary row data in public research artifacts.

IX. Future Work

1) Evaluate cross-schema generalization: score the trained adapters on an external Spider subset and on freshly generated synthetic schemas unseen during training, with confidence intervals over multiple seeds.
2) Add schema-retrieval / semantic-layer conditioning so models generalize to unseen warehouses.
3) Train preference or DPO stages on human-corrected SQL from private capture logs.
4) Evaluate multi-skill tool-calling traces rather than single-turn JSON only.
5) Study distillation from a large hosted orchestrator into a small on-prem router+SQL specialist under strict privacy constraints.

Acknowledgment

This work began as coursework at Florida International University. The experimental warehouse, questions, and labels are synthetic. The research is motivated by enterprise BI agent architectures but does not use proprietary employer data.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. [Online]. Available: https://openreview.net/pdf?id=nZeVKeeFYf9

[2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[3] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, Brussels, Belgium, 2018, pp. 3911–3921. [Online]. Available: https://arxiv.org/abs/1809.08887

[4] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Proc. NeurIPS*, 2023. [Online]. Available: https://arxiv.org/abs/2304.11015

[5] F. Lei *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *Proc. ICLR*, 2025. [Online]. Available: https://spider2-sql.github.io/

[6] Z. Han *et al.*, "Parameter-efficient fine-tuning for large models: A comprehensive survey," 2024. [Online]. Available: https://arxiv.org/abs/2403.14608

[7] S. Wang *et al.*, "FinLoRA: Finetuning quantized financial large language models using LoRA," 2024. [Online]. Available: https://arxiv.org/abs/2412.11378